Collaborative Study Material & Quiz Platform

Detailed Work Division for a 4-Member Software Engineering Team

Project Process: Jira + GitHub + MERN Stack

Team Labels: Member 1, Member 2, Member 3, Member 4

Basis: Revised SRS with 17 Functional Requirements, 5 NFRs and 5 Security Requirements

Purpose: Clear ownership of development, testing, Jira evidence, GitHub contributions, integration and final delivery.

1. Team Responsibility Overview

The work is divided by functional modules, but every member must contribute code, maintain Jira issues, use GitHub feature branches and pull requests, participate in testing, and help with final integration. The division below avoids making any one member only a documentation or management contributor.

Member	Primary Responsibility	Core Modules	Cross-Team Responsibility
Member 1	Authentication + Study Material Management	Register, Login, Logout, PDF Upload, Material Search, View/ Download	Frontend consistency, browser compatibility and usability
Member 2	Question Bank + Quiz Creation	Add/Search MCQs, Create Quiz, Search/View Quizzes	Jira coordination and feature/ API integration
Member 3	Quiz Attempt + Evaluation + Performance	Attempt Quiz, Auto Evaluation, Results, History, Subject-wise Average	QuizAttempt data model, evaluation testing, cross-test admin/security
Member 4	Backend Foundation + Admin + Security + Deployment	Common backend setup, MongoDB connection, Admin Moderation, Security, Deployment	GitHub coordination, integration, performance/load testing

1.1 Database and API ownership rule

> **Important correction:** Member 4 owns the shared backend foundation and MongoDB connection, but does not build every member's database model or controller. Each feature owner builds the model/API logic for their own module. This prevents Member 4 from becoming a bottleneck and gives every member meaningful backend/GitHub contribution.

Technical Area	Primary Owner
MongoDB connection, Express app structure, shared middleware	Member 4
User + Material schemas/models and related APIs	Member 1
Question + Quiz schemas/models and related APIs	Member 2
QuizAttempt/Result schema/model and evaluation/performance APIs	Member 3
Security conventions, role checks, integration and deployment	Member 4

2. Member 1 - Authentication and Study Material Management

Primary Role: Build the student-facing authentication and study-material flow, including the related frontend, feature APIs, models and usability/browser testing.

2.1 Functional Requirements Owned

SRS ID	Responsibility	Expected Working Output
CSMQ-F-001	User registration	Student can create an account using valid details.
CSMQ-F-002	Login + invalid-login handling	Valid login succeeds; invalid credentials show a clear error.
CSMQ-F-004	Logout	Session/token is cleared and protected pages are inaccessible.
CSMQ-F-010	Upload PDF study material	Valid PDF within configured size is stored and listed.
CSMQ-F-012	Search/filter study material	Students can find resources by keyword or subject.
CSMQ-F-013	View/download study material	Stored PDF can be opened or downloaded.

2.2 Detailed Development Tasks

- Set up the React screens for Registration, Login, Logout, Student Dashboard and Study Materials.
- Implement form validation for required fields, email format and user-friendly error messages.
- Implement or integrate authentication APIs for register/login/logout with the shared middleware from Member 4.
- Create the User and Material schema/model fields required by this module.
- Build the PDF upload form with title, subject, description and file selection.
- Validate PDF-only uploads and the configured maximum file size on both frontend and backend.
- Build material list, search and subject filter UI.
- Implement View/Download actions and handle missing/deleted material gracefully.
- Keep loading states, empty states and errors consistent across the student-facing screens.

2.3 Main Components and APIs

Area	Work	Completion Check
React UI	Register/Login/Logout, Upload Material, Material List/ Search/View	Student completes full flow without manual DB changes.
User Model/API	User model + register/login/logout integration	Valid session works; invalid login is rejected.
Material Model/ API	Material metadata, upload/list/search/view/download	Uploaded PDF can later be found and opened.
Validation	Email, required fields, PDF type, file size	Invalid input is rejected with clear messages.

2.4 NFR Ownership

- **CSMQ-NF-004 - Browser Compatibility:** verify the core student flow on recent Chrome, Edge and Firefox.
- **CSMQ-NF-005 - Usability:** keep navigation, success messages, validation messages and error messages clear.

2.5 Jira Work

- Create/update Jira stories for registration, login, logout, PDF upload, material search and view/download.

- Move issues through: TO DO -> IN PROGRESS -> CODE REVIEW -> TESTING -> DONE.
- Add pull-request links, screenshots or test evidence before closing each issue.
- Cross-test Member 3's quiz attempt/evaluation flow and log reproducible defects.

2.6 GitHub Work

- Use one feature branch per Jira issue, e.g. `feature/CSMQ-10-material-upload`.
- Use meaningful commits, e.g. `CSMQ-10 Add PDF upload validation`.
- Open a PR and request review before merge; do not push complete features directly to main.

2.7 Sprint Plan

Sprint	Member 1 Work	Evidence
0	React routing, shared layout, API client skeleton	Initial branch, Jira tasks, base screens
1	Registration, login, logout, User model/API, PDF upload	Working auth/upload PRs
2	Material listing, search/filter, view/download	End-to-end material flow
3	Browser/usability fixes and integration polishing	Compatibility checklist + resolved bugs
4	Cross-testing and demo preparation	Test evidence + final screenshots

3. Member 2 - Question Bank, Quiz Creation and Jira Coordination

Primary Role: Build the content-authoring side of the system: question bank and quiz creation/search. Also coordinate Jira quality without becoming the only person responsible for Jira updates.

3.1 Functional Requirements Owned

SRS ID	Responsibility	Expected Working Output
CSMQ-F-020	Add/classify MCQ question	MCQ with options, correct answer, subject, topic and difficulty is stored.
CSMQ-F-022	Search/filter question bank	Questions are searchable/filterable by subject, topic or difficulty.
CSMQ-F-030	Create quiz using question-bank questions	Quiz with title, subject and selected questions is stored.
CSMQ-F-032	Search/view available quizzes	Students can find and open available quizzes.

3.2 Detailed Development Tasks

- Create Question and Quiz schemas/models needed by this module.
- Build Add Question UI with MCQ text, options, correct answer, subject, topic and difficulty.
- Validate required fields and ensure exactly one correct answer is stored for the MCQ.
- Build question-bank listing, search and filters.
- Build Create Quiz UI with title, subject and question selection from the question bank.
- Prevent empty quizzes and display validation messages.
- Build quiz list/search/detail or preview screen.
- Coordinate the quiz data structure with Member 3 so quiz attempts and evaluation can consume it directly.
- **Security rule:** do not expose `correctAnswer` to the student before submission. The student-facing quiz API should return question text and options only.

3.3 Main Components and APIs

Area	Work	Completion Check
Question Model/UI	Create question, store classification fields, browse/filter	Created question can be found by filters.
Question APIs	Create/list/filter endpoints	Stored metadata matches UI selection.
Quiz Model/UI	Create quiz, select questions, list/search quizzes	Quiz is stored with intended question references.
Quiz APIs	Create/list/detail/search endpoints	Created quiz is retrievable and attemptable by Member 3 flow.

3.4 Jira Coordinator Responsibilities

- Create and maintain Epics for Authentication, Materials, Question Bank, Quiz, Performance, Admin/Security, Testing and Documentation.
- Ensure each issue has an owner, acceptance criteria, priority and realistic sprint assignment.
- Check that blocked tasks and dependencies are visible on the board.
- Verify that PR links/test evidence are attached before issues move to DONE.

- Do not update everyone's work for them - each member must maintain their own Jira issues.

3.5 GitHub Work

- Create feature branches tied to Jira issue IDs and use meaningful commits.
- Open PRs for question-bank and quiz-creation changes and request review.
- Document the Question/Quiz API contract if Member 3 depends on it.

3.6 Testing Responsibility

- Test valid/invalid MCQ creation.
- Test question filters and quiz creation with selected questions.
- Test quiz list/search/detail.
- Cross-test Member 1's authentication/material module and report defects in Jira.

3.7 Sprint Plan

Sprint	Member 2 Work	Evidence
0	Jira board/Epics, Question/Quiz model design and UI wireframe	Backlog + initial schemas/wireframe
1	Question model/API, Add MCQ and question filtering	Question Bank PR + tests
2	Quiz model/API, create quiz, select questions, quiz search/view	Working quiz-authoring flow
3	Integration with Member 3, Jira cleanup, edge-case fixes	Stable data contract + clean board
4	Cross-testing and final Jira evidence	Closed issues with PR/test links

4. Member 3 - Quiz Attempt, Evaluation, Results and Performance

Primary Role: Build the student quiz execution pipeline from attempt to result/history/performance, including QuizAttempt data storage and server-side evaluation logic.

4.1 Functional Requirements Owned

SRS ID	Responsibility	Expected Working Output
CSMQ-F-033	Attempt available quiz	Student can answer MCQs in an available quiz.
CSMQ-F-034	Automatic MCQ evaluation	Backend calculates correct score from submitted responses.
CSMQ-F-035	Display quiz result	Score/percentage is shown after successful submission.
CSMQ-F-036	Quiz attempt history	Previous attempts are stored and viewable by the correct user.
CSMQ-F-040	Subject-wise average score	Average score for each subject is calculated from previous attempts.

4.2 Detailed Development Tasks

- Create the QuizAttempt/Result schema/model with user, quiz, responses or score data, percentage and timestamp as required.
- Build the quiz attempt screen showing question text and options received from Member 2's quiz API.
- Maintain selected answers and handle unanswered questions predictably.
- Submit the student's responses to the backend for evaluation.
- **Submission rule:** each individual quiz attempt should be accepted only once to prevent accidental duplicate submissions. This does not necessarily prohibit a student from starting a later repeat attempt if the system permits it.
- Evaluate answers server-side by comparing submitted responses with stored correct answers.
- Calculate raw score and percentage consistently.
- Store completed attempts and ensure they persist after restart/reconnect.
- Build result and quiz-history screens.
- Calculate subject-wise average as the sum of attempt percentages for that subject divided by the number of attempts.
- Handle subjects with no attempts and avoid divide-by-zero errors.

4.3 Main Components and APIs

Area	Work	Completion Check
Quiz Attempt UI	MCQ display, answer selection, submission confirmation	Student can complete and submit an attempt.
Evaluation API	Receive responses and calculate score server-side	Known test quiz gives expected score.
QuizAttempt Model	Persist user/quiz/result/timestamp	History remains after restart.
History UI/API	List logged-in student's prior attempts	Only correct user's attempts are shown.
Performance API/ UI	Compute and display subject-wise average	Manual calculation matches displayed result.

4.4 NFR and Testing Responsibilities

- Support **CSMQ-NF-002 - Data Persistence** with Member 4 by verifying attempts/results remain correct after application restart and database reconnect.

- Cross-test Member 4's Admin + Security module, especially student attempts to access admin-only operations.
- Test full-score, zero-score, mixed-score, unanswered-question, repeated-request and no-history cases.

4.5 Jira and GitHub Work

- Create/update Jira issues for quiz attempt, evaluation, result, history and performance.
- Attach test evidence showing expected vs actual scores.
- Use feature branches and PRs; request review from another member before merge.
- Coordinate API changes with Members 2 and 4 before changing shared request/response structures.

4.6 Sprint Plan

Sprint	Member 3 Work	Evidence
0	Agree quiz-attempt/result data contract with Members 2 and 4	Documented request/response fields
1	QuizAttempt model + attempt UI prototype + evaluation logic prototype	Local working prototype
2	Submission, server-side evaluation and result display	End-to-end attempt flow
3	History, subject average and persistence testing	Stored history + correct averages
4	Cross-test Member 4, edge cases and demo preparation	Security/admin defects logged and resolved

5. Member 4 - Backend Foundation, Admin, Security, Deployment and GitHub

Primary Role: Own the shared technical foundation, security controls, administrator moderation, integration, deployment and GitHub process. Member 4 supports other modules without taking over their feature-specific models/controllers.

5.1 Functional and Shared Requirements Owned

Requirement	Responsibility	Expected Working Output
CSMQ-F-041	Admin moderate study materials	Admin can remove inappropriate/invalid material; students cannot.
CSMQ-F-042	Admin moderate questions/quizzes	Admin can remove inappropriate question/quiz; students cannot.
Shared Backend	Express structure, MongoDB connection, middleware, config	All feature modules can integrate into one stable backend.
Security	Password hashing, authentication, RBAC, validation, HTTPS	Five SRS security requirements are implemented/ tested.
Integration/ Deployment	Environment configuration, build integration and deployment	Team can run or demonstrate a single integrated system.

5.2 Detailed Development Tasks

- Initialize the shared Express application structure, environment configuration, route organization and common error handling.
- Configure MongoDB connection and common connection/error utilities.
- Provide reusable authentication, authorization and request-validation middleware.
- Support Members 1-3 with schema/API integration while leaving feature model/controller ownership with those members.
- Implement admin-only moderation endpoints/screens needed to remove study materials, questions and quizzes.
- Ensure student accounts cannot call admin-only routes.
- Hash passwords using a suitable library such as bcrypt; never store plain-text passwords.
- Implement JWT/session authentication and protected-route middleware.
- Implement role-based authorization for administrator-only actions.
- Validate/sanitize backend input and uploaded-file metadata.
- Configure deployment environment variables and HTTPS where deployed.
- Prepare final setup/deployment instructions and perform the integrated smoke test.

5.3 NFR Ownership

NFR	Member 4 Responsibility
CSMQ-NF-001 - Performance	Measure that at least 90% of standard requests/pages meet the 3-second target under normal test conditions.
CSMQ-NF-002 - Data Persistence	Work with Member 3 to verify stored users/materials/questions/quizzes/results survive restart/reconnect.
CSMQ-NF-003 - Scalability	Run a defined load test for 50 concurrent users without critical functional failure.

5.4 Security Requirements Owned

SRS Security ID	Implementation Responsibility
CSMQ-SR-001	Secure password hashing; no plain-text password storage.
CSMQ-SR-002	Authentication required for protected functionality.
CSMQ-SR-003	Role-based access control for administrator functionality.
CSMQ-SR-004	Backend input validation/sanitization.
CSMQ-SR-005	HTTPS for sensitive client-server communication in deployed environment.

5.5 GitHub Coordinator Responsibilities

- Create/maintain repository structure, `.gitignore`, branch conventions and pull-request workflow.
- Maintain `main` as the stable branch and optionally `develop` as integration branch.
- Help resolve merge conflicts and ensure the integrated branch remains buildable.
- Do not become the only person merging or contributing code - every member must create branches, commits and PRs.
- Prepare final release/tag and README setup/deployment instructions.

5.6 Testing Responsibility

- Test authentication middleware, admin role checks, protected routes and invalid/malicious input handling.
- Test moderation success/failure cases.
- Run performance/load smoke tests for NFR-001 and NFR-003.
- Cross-test Member 1's authentication/material flow.
- Member 3 independently cross-tests Member 4's Admin + Security module.

5.7 Sprint Plan

Sprint	Member 4 Work	Evidence
0	Repo, backend, MongoDB connection, env config, branch rules	Working base repository + setup notes
1	Auth middleware, hashing, role foundations, Member 1 integration support	Security foundation PRs
2	Common integration support, validation, protected routes	Integrated backend passes smoke tests
3	Admin moderation + persistence/performance/load tests	Admin features + NFR evidence
4	Deployment, final integration, release/tag and README	Demo-ready integrated release

6. Cross-Testing Matrix

Each major module should be tested by someone other than its primary developer. This provides stronger Jira evidence and avoids self-verification only.

Module	Primary Developer	Independent Tester
Authentication + Study Materials	Member 1	Member 2
Question Bank + Quiz Creation	Member 2	Member 3
Quiz Attempt + Evaluation + Performance	Member 3	Member 1
Admin + Security	Member 4	Member 3
Full Integration	All Members	All Members

7. Shared Jira Workflow

Stage	Meaning
TO DO	Task is ready but has not started.
IN PROGRESS	Assignee is actively implementing it.
CODE REVIEW	PR is open and another member is reviewing it.
TESTING	Acceptance criteria and test cases are being verified.
DONE	Code is merged, tested, evidence attached and no critical issue remains.

7.1 Suggested Jira Epics

Epic	Primary Owner
Project Setup & Authentication	Member 1 + Member 4
Study Material Management	Member 1
Question Bank	Member 2
Quiz Creation	Member 2
Quiz Attempt & Evaluation	Member 3
Performance Analysis	Member 3
Administration & Security	Member 4
Testing & Integration	All Members
Documentation & Deployment	All Members

8. Shared GitHub Workflow

> **Recommended flow:** Feature Branch -> Pull Request -> Code Review -> Merge to develop/integration branch -> Testing -> Merge to main.

- No member should develop complete features directly on `main`.
- Use Jira-linked branch names such as `feature/CSMQ-33-quiz-attempt`.

- Use meaningful commits such as `CSMQ-34 Add server-side MCQ evaluation`.
- Every PR should be reviewed by at least one other member.
- Attach PR links back to Jira issues.

8.1 Suggested Review Pairing

Developer	Suggested Reviewer
Member 1	Member 2
Member 2	Member 3
Member 3	Member 4
Member 4	Member 1 or Member 3 depending on module

9. Combined Sprint Plan

Sprint	Member 1	Member 2	Member 3	Member 4
0	React setup, routing	Jira setup, question/quiz design	Quiz data contract planning	Repo, backend, MongoDB connection
1	Auth + PDF upload	Question Bank	QuizAttempt model + prototype	Auth/security foundation
2	Material search/view/ download	Quiz creation/search	Attempt/evaluation/result	Shared integration + validation
3	Compatibility/usability polish	Integration/Jira cleanup	History + subject averages	Admin + NFR tests
4	Cross-test/demo	Cross-test/Jira evidence	Cross-test admin/ security/demo	Deployment + final release

10. Definition of Done for Any Jira Issue

1. Requirement/acceptance criteria are understood.
2. Implementation is complete on a feature branch.
3. Meaningful commits are pushed to GitHub.
4. A Pull Request is opened.
5. Another team member reviews the code.
6. Review comments are addressed.
7. Feature is merged to the integration branch.
8. Acceptance test passes.
9. Jira issue contains PR/test evidence.
10. No critical bug remains before the issue moves to DONE.

11. Final Ownership Summary

Member	Final Ownership
Member 1	Authentication + Study Materials + User/Material models/APIs + browser/usability checks.
Member 2	Question Bank + Quiz Creation/Search + Question/Quiz models/APIs + Jira coordination.
Member 3	Quiz Attempt + Auto Evaluation + Result/History + Performance + QuizAttempt model/API + cross-test Admin/Security.
Member 4	Shared backend foundation + MongoDB connection + middleware + Admin + Security + NFR performance/load tests + Deployment + GitHub coordination.
All Members	Jira updates, GitHub branches/commits/PRs, cross-testing, integration, documentation and final demo.

This division is intentionally aligned with the revised 17-FR SRS and is designed to keep the implementation manageable while giving all four members visible software-engineering contributions.